SOFTWARE ENGINEERING

Assignment 3 : Detailed Design

Collaborative Real-Time Coding Platform

Submitted by:

Usman Raza Alvi (BSCE23047)

Mufti Abdul Tawwab (BSCE23054)

Hassan Mehmood (BSCE23031)

Instructor :

Muhammad Umair Shoaib

Table of Contents

1. INTRODUCTION

1.1 Project Overview

The Collaborative Real-Time Coding Platform is a web-based system designed to enable multiple users to collaboratively solve coding problems in real-time. The system supports shared code editing, code execution, chat communication, room-based collaboration, and performance tracking. The platform integrates modern web technologies and real-time synchronization mechanisms to simulate a collaborative development environment similar to professional online IDEs.

1.2 Project Overview

This assignment focuses on the detailed design phase of the software project. It extends the Requirements Engineering and Scrum-based architectural work completed in previous assignments by developing structural and dynamic UML models of the system.

The report includes:

- UML Class Diagram
- Design Pattern Application
- Sequence Diagrams
- State Diagram
- Activity Diagram
- Detailed workflow explanations
- GitHub repository integration

2. STRUCTURAL MODEL

2.1 UML Class Diagram

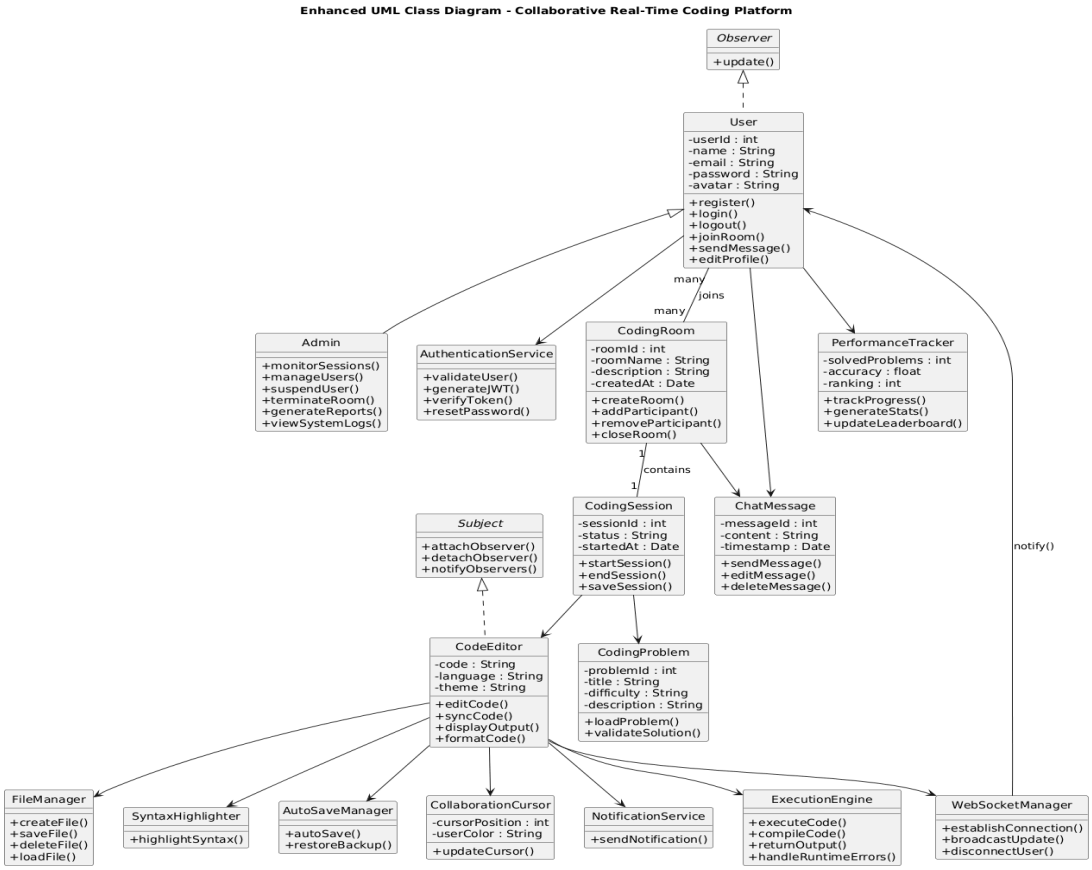

Diagram Explanation

The UML class diagram models the core entities and services of the collaborative coding platform. The system includes user management, collaborative coding rooms, shared code editors, authentication services, code execution services, chat communication, and performance tracking modules.

The diagram also incorporates advanced IDE-related components such as syntax highlighting, auto-saving, collaboration cursors, and notification management to support real-time collaborative development.

The relationships among classes represent inheritance, associations, and service interactions required for system functionality.

2.2 Design Pattern Used – Observer Pattern

Observer Pattern Justification

The Observer Pattern is implemented in the real-time collaboration module of the system. The CodeEditor acts as the Subject, while connected users act as Observers. Whenever a user modifies the shared code editor, the system notifies all connected users through the WebSocketManager, ensuring real-time synchronization of code updates.

This design pattern is suitable because the collaborative coding platform requires multiple users to receive immediate updates whenever shared code changes occur

3. DYNAMIC MODEL

3.1 Sequence Diagram 1 – Run Code & Real-Time Collaboration

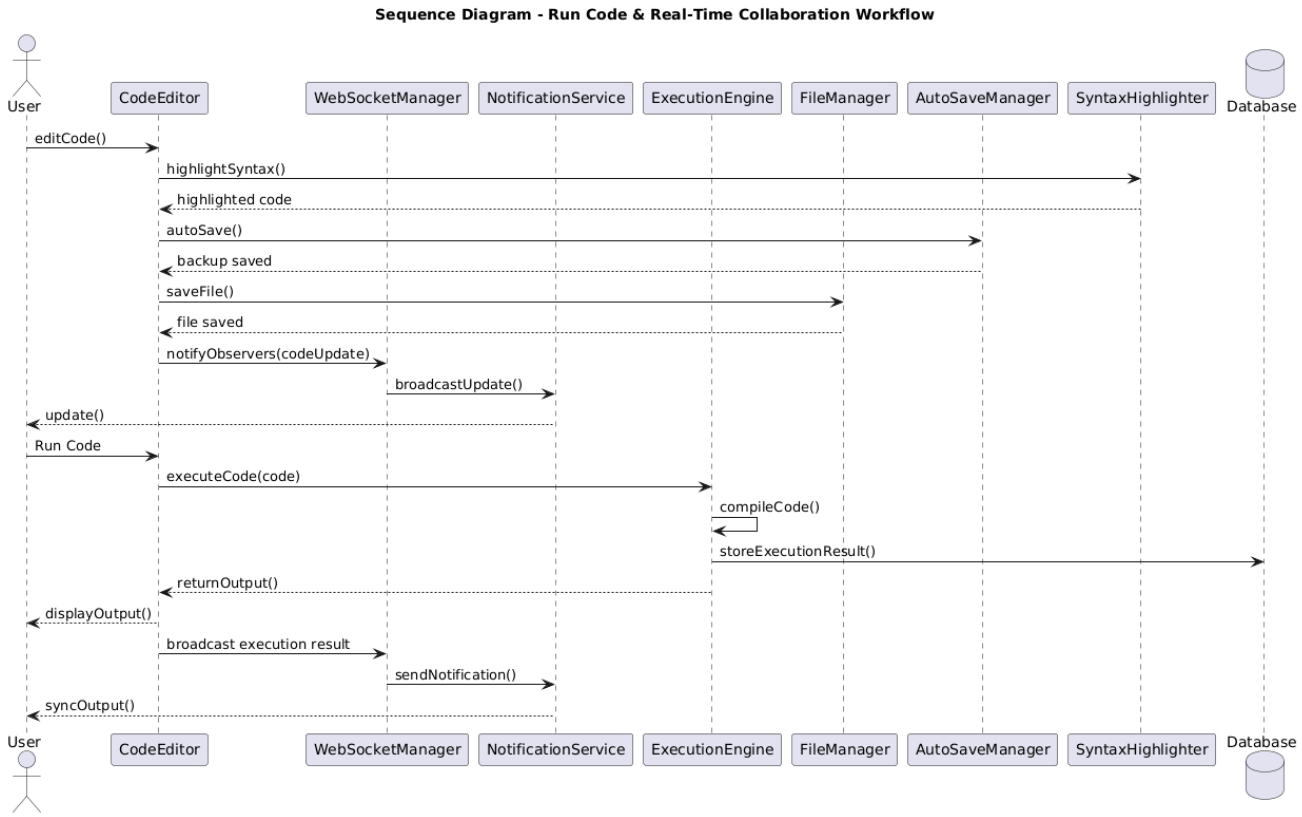

Diagram Explanation

This sequence diagram represents the workflow of collaborative code editing and code execution. Users interact with the CodeEditor, which supports IDE functionalities such as syntax highlighting, auto-saving, and file management.

Real-time synchronization is handled using the Observer Pattern through the WebSocketManager and NotificationService. When code execution is triggered, the ExecutionEngine compiles and executes the code, stores execution results, and synchronizes outputs across all connected users.

3.2 Sequence Diagram 2 – Join Room & Session Initialization

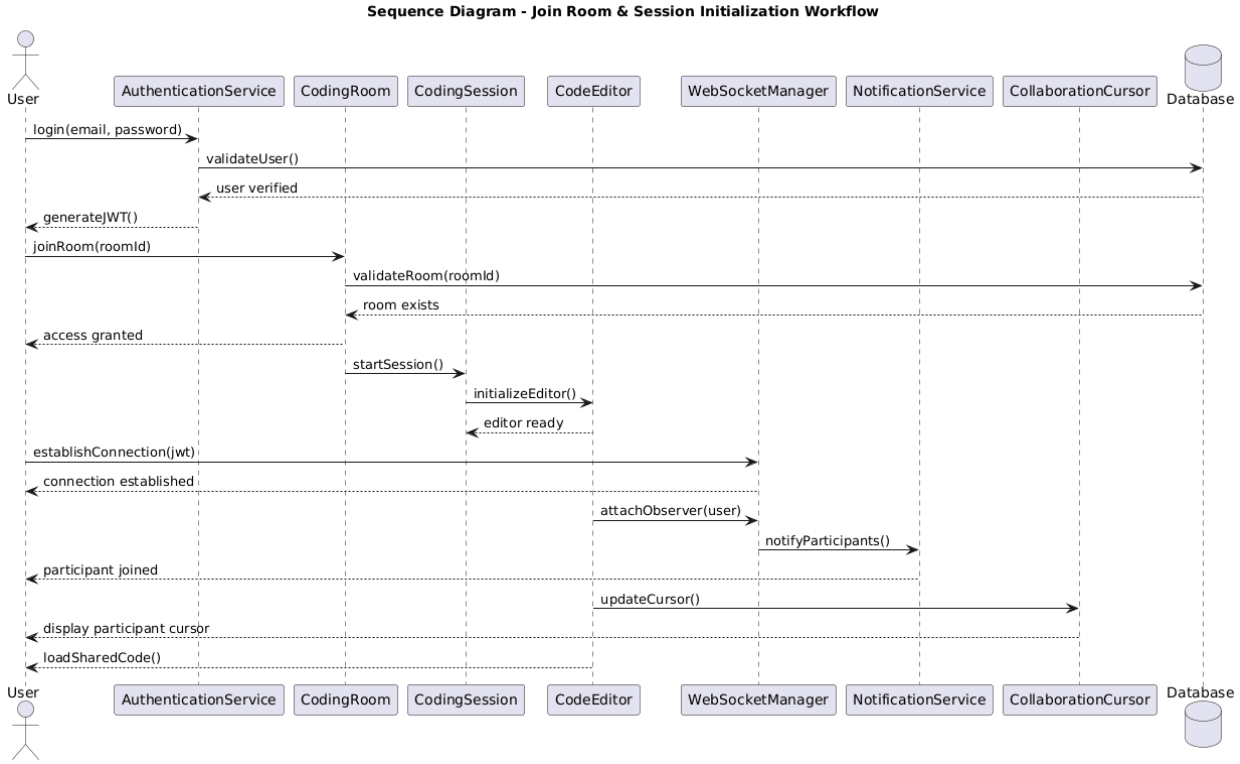

Diagram Explanation

This sequence diagram models the workflow for joining a collaborative coding room. The user is authenticated through the AuthenticationService, after which the room and session are initialized.

The WebSocketManager establishes real-time communication and attaches users as observers to enable synchronization. Additional collaboration services such as participant notifications and cursor synchronization are also initialized before collaborative editing begins.

3.2 State Diagram – Coding Session Lifecycle

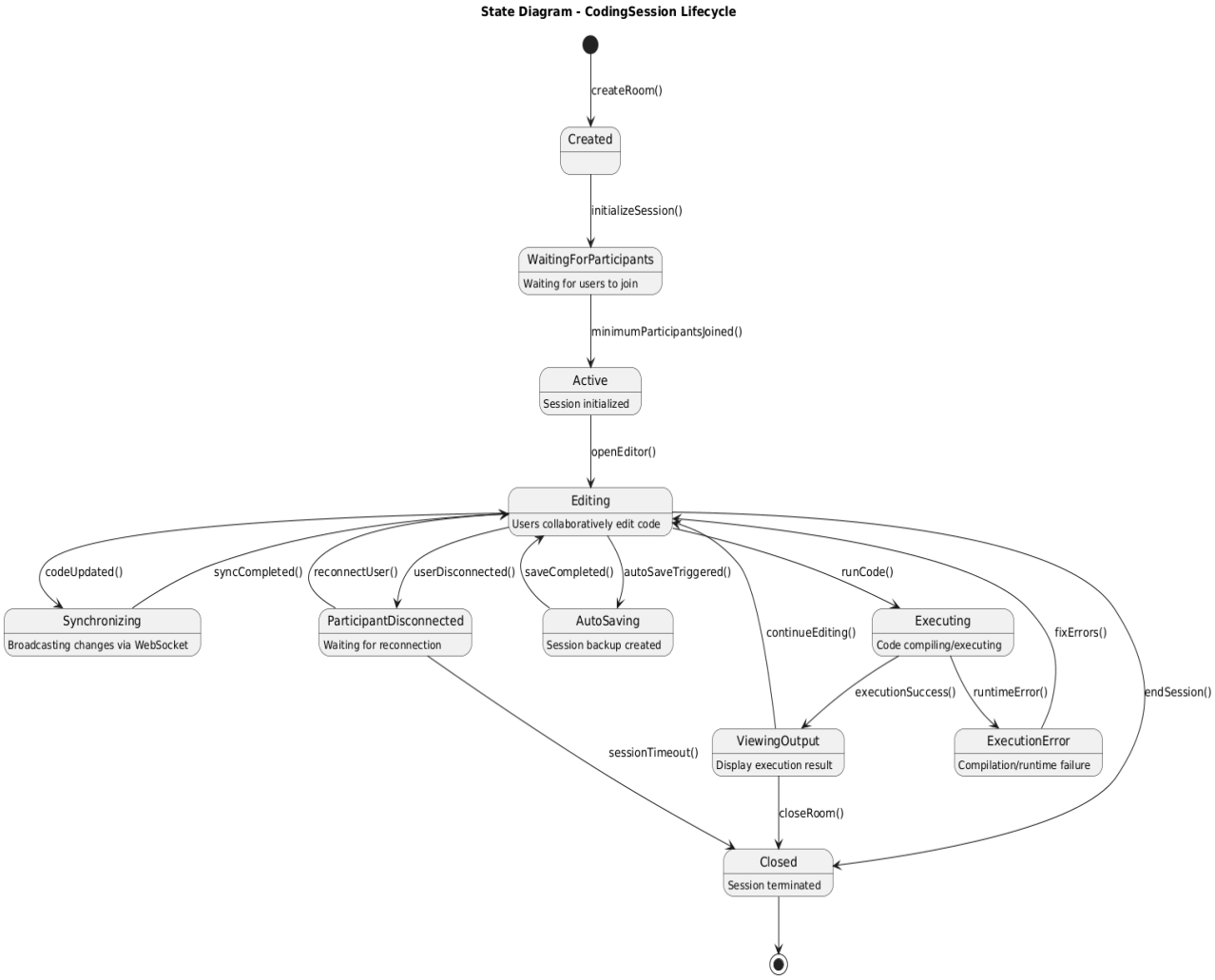

Diagram Explanation

The state diagram represents the lifecycle of a collaborative coding session. The session transitions through multiple states including room creation, participant waiting, active collaboration, code editing, synchronization, execution, auto-saving, and session termination.

The diagram also models runtime errors and participant disconnection handling to ensure fault tolerance and recovery during collaborative sessions.

3.4 Activity Diagram – Collaborative Coding Workflow

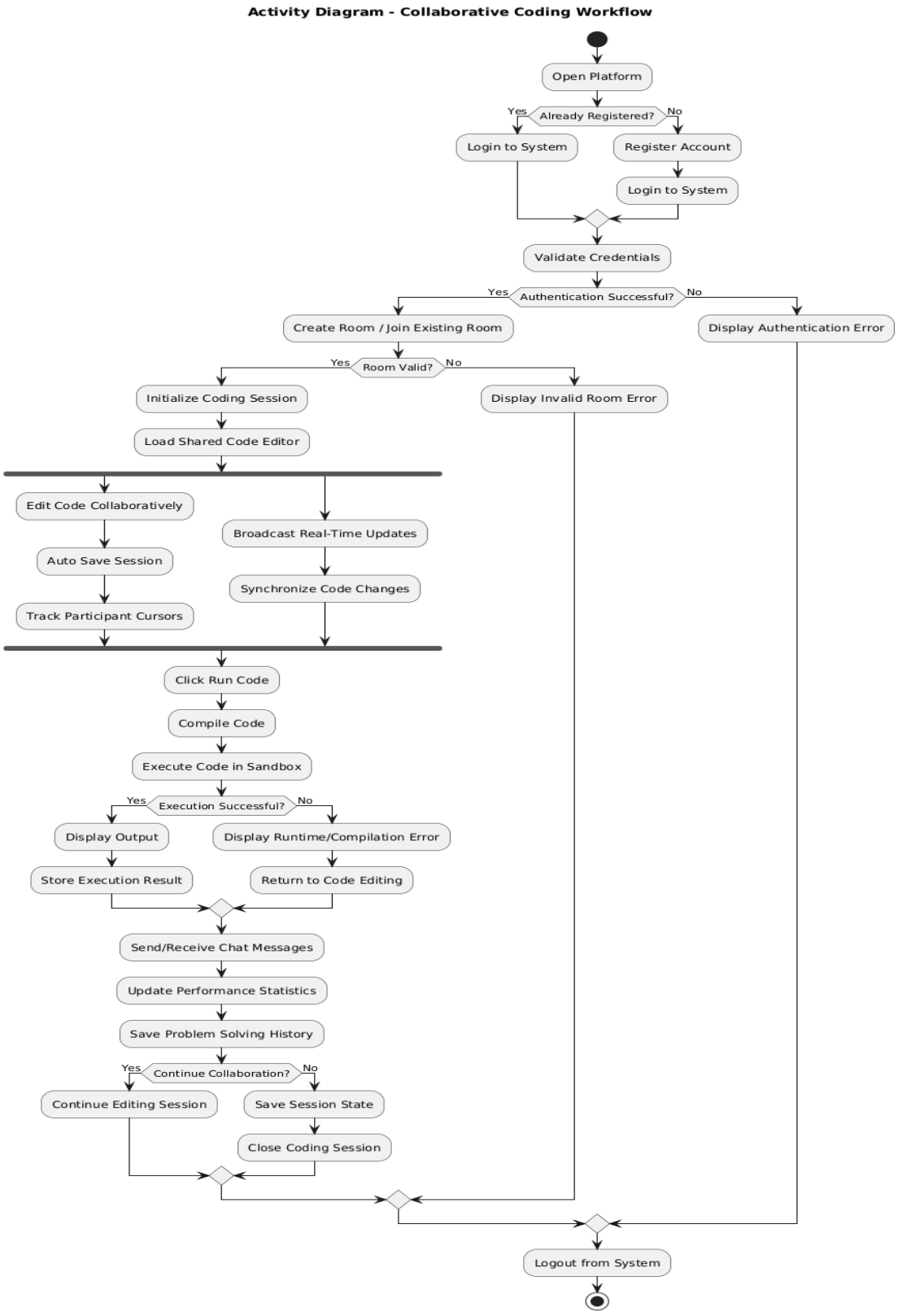

Diagram Explanation

The activity diagram models the overall workflow of the collaborative coding platform, beginning from user authentication and room management to real-time collaborative editing and code execution.

The workflow includes parallel activities such as auto-saving, participant cursor tracking, and real-time synchronization. Additional functionalities including communication, performance tracking, and session management are also represented.

4. SYSTEM WORKFLOW & DESIGN ANALYSIS

4.1 System Workflow

The system begins with user authentication through the AuthenticationService. Once authenticated, users can create or join collaborative coding rooms. Each room initializes a coding session and a shared code editor.

Users collaboratively edit code in real-time using WebSocket-based synchronization. The system supports IDE-like functionalities such as syntax highlighting, auto-saving, collaboration cursors, and notification management.

When code execution is requested, the ExecutionEngine compiles and executes the code inside a secure sandbox environment and returns the output to all connected users. User activities and problem-solving performance are continuously tracked throughout the session.

4.2 Architectural Consistency

The detailed design models remain fully consistent with the layered architecture and Scrum-based implementation developed in previous assignments. The structural and dynamic models accurately reflect the system functionalities identified during requirements engineering and sprint planning activities.

5. GITHUB REPOSITRY

All UML models, diagrams, and project-related artifacts were uploaded to the project GitHub repository as required.

Repository Link

Github link

6. CONCLUSION

This assignment successfully developed the detailed structural and dynamic models of the Collaborative Real-Time Coding Platform. UML diagrams including class diagrams, sequence diagrams, state diagrams, and activity diagrams were created to represent both the static and behavioral aspects of the system.

The implementation of the Observer Pattern improved the real-time synchronization architecture of the platform, while the dynamic models demonstrated collaborative workflows, code execution processes, and session management behaviors.

Overall, the assignment demonstrates a complete transition from requirements engineering and Scrum-based planning toward detailed software system design and analysis.

7. APPENDIX

Additional Design Artifacts

The following additional resources and high-resolution diagrams are provided as compressed images due to size and readability constraints:

- High-Resolution UML Diagrams
- Additional Architecture Diagrams
- Sprint Artifacts and Reports

Access Link

Drive Link